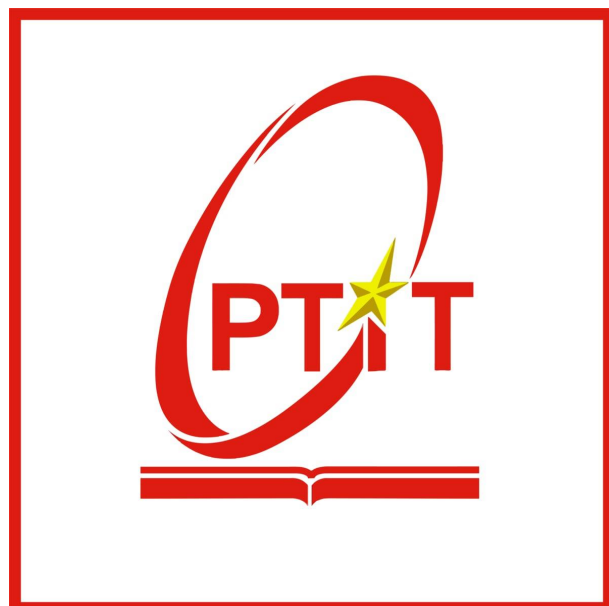


**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG  
KHOA CÔNG NGHỆ THÔNG TIN**

**HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU**



**BÁO CÁO BÀI TẬP LỚN- NHÓM 6**

**HỆ THỐNG QUẢN LÝ GIẢI ĐUA XE CÔNG THỨC F1**

**Giảng viên hướng dẫn:** PGS.TS. Nguyễn Trọng Khánh

**Nhóm sinh viên thực hiện:** Nguyễn Văn Hiếu - B23DCCE033

Phạm Nhật Minh - B23DCCE066

Lê Minh Đức - B23DCVT095

Hà Nội, Tháng 4/2026

# Mục lục

|          |                                                              |           |
|----------|--------------------------------------------------------------|-----------|
| <b>1</b> | <b>CHƯƠNG 1: GIỚI THIỆU</b>                                  | <b>6</b>  |
| 1.1      | Tổng quan về DBMS                                            | 6         |
| 1.2      | Vai trò của DBMS trong hệ thống thông tin                    | 6         |
| 1.3      | Giới thiệu bài toán — F1 Championship Management System      | 6         |
| 1.4      | Mục tiêu báo cáo                                             | 7         |
| <b>2</b> | <b>CHƯƠNG 2: CƠ SỞ LÝ THUYẾT</b>                             | <b>8</b>  |
| 2.1      | Khái niệm DBMS                                               | 8         |
| 2.2      | Các mô hình dữ liệu                                          | 8         |
| 2.3      | Kiến trúc 3 mức (Three-Level Architecture)                   | 8         |
| 2.4      | Các khái niệm cơ bản                                         | 8         |
| <b>3</b> | <b>CHƯƠNG 3: MÔ TẢ BÀI TOÀN &amp; PHÂN TÍCH YÊU CẦU</b>      | <b>10</b> |
| 3.1      | Mô tả hệ thống                                               | 10        |
| 3.2      | Người dùng hệ thống                                          | 10        |
| 3.3      | Yêu cầu chức năng (Functional Requirements)                  | 10        |
| 3.4      | Yêu cầu phi chức năng (Non-Functional Requirements)          | 11        |
| <b>4</b> | <b>CHƯƠNG 4: LOGIC NGHIỆP VỤ</b>                             | <b>12</b> |
| 4.1      | Quy trình nghiệp vụ tổng thể                                 | 12        |
| 4.2      | Business Rules (Quy tắc nghiệp vụ)                           | 12        |
| 4.3      | Trigger — Ràng buộc nghiệp vụ tại tầng Database              | 13        |
| 4.3.1    | Trigger 1: trg_limit_2_riders                                | 13        |
| 4.3.2    | Trigger 2 & 3: trg_check_time_insert / trg_check_time_update | 14        |
| 4.4      | Transaction — Đảm bảo tính nguyên tử                         | 14        |
| 4.5      | Stored Procedure — sp_calculate_points                       | 15        |
| 4.6      | Views — Bảng ảo tổng hợp BXH                                 | 16        |
| <b>5</b> | <b>CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU</b>                      | <b>18</b> |
| 5.1      | Mô hình thực thể — liên kết (ERD)                            | 18        |
| 5.2      | Mô tả chi tiết các thực thể                                  | 18        |
| 5.3      | Mô hình quan hệ                                              | 19        |
| 5.4      | Chuẩn hóa (Normalization)                                    | 19        |
| 5.4.1    | 1NF — First Normal Form                                      | 19        |
| 5.4.2    | 2NF — Second Normal Form                                     | 19        |
| 5.4.3    | 3NF — Third Normal Form                                      | 19        |
| 5.4.4    | BCNF — Boyce-Codd Normal Form                                | 19        |
| <b>6</b> | <b>CHƯƠNG 6: TRIỂN KHAI</b>                                  | <b>21</b> |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| 6.1      | Môi trường và công nghệ . . . . .          | 21        |
| 6.2      | Schema — Tạo cấu trúc bảng (DDL) . . . . . | 21        |
| 6.3      | Nạp dữ liệu mẫu (DML — INSERT) . . . . .   | 22        |
| 6.4      | Các câu truy vấn SQL quan trọng . . . . .  | 23        |
| 6.5      | Index — Tối ưu hóa hiệu năng . . . . .     | 24        |
| <b>7</b> | <b>CHƯƠNG 7: ĐÁNH GIÁ</b>                  | <b>25</b> |
| 7.1      | Đánh giá thiết kế cơ sở dữ liệu . . . . .  | 25        |
| 7.1.1    | Điểm mạnh . . . . .                        | 25        |
| 7.2      | Hạn chế hiện tại . . . . .                 | 25        |
| 7.3      | Khả năng mở rộng . . . . .                 | 26        |
| <b>8</b> | <b>CHƯƠNG 8: KẾT LUẬN</b>                  | <b>27</b> |
| 8.1      | Tóm tắt kết quả . . . . .                  | 27        |
| 8.2      | Bài học rút ra . . . . .                   | 27        |
| 8.3      | Hướng phát triển . . . . .                 | 27        |
|          | <b>Tài Liệu Tham Khảo</b>                  | <b>28</b> |

# Danh sách hình vẽ

|     |                                                             |    |
|-----|-------------------------------------------------------------|----|
| 4.1 | Vòng đời dữ liệu của một chặng đua trong hệ thống . . . . . | 12 |
| 4.2 | Luồng xử lý của Trigger trg_limit_2_riders . . . . .        | 13 |
| 4.3 | Luồng Transaction của Module 2 — Nhập kết quả . . . . .     | 15 |
| 5.1 | Sơ đồ ERD của hệ thống F1 Championship Management . . . . . | 18 |

# Danh sách bảng

|     |                                                      |    |
|-----|------------------------------------------------------|----|
| 3.1 | Các module của hệ thống . . . . .                    | 10 |
| 4.1 | Hệ thống tính điểm F1 . . . . .                      | 13 |
| 5.1 | Bảng CHAMPIONSHIPS — Mùa giải . . . . .              | 18 |
| 5.2 | Bảng CONTRACTS — Hợp đồng tay đua — đội . . . . .    | 18 |
| 5.3 | Bảng RESULTS — Kết quả thi đấu . . . . .             | 19 |
| 6.1 | Bảng công nghệ sử dụng . . . . .                     | 21 |
| 6.2 | Chiến lược Index của hệ thống . . . . .              | 24 |
| 7.1 | Tổng hợp các đối tượng DBMS trong hệ thống . . . . . | 25 |

# Listings

|     |                                                                       |    |
|-----|-----------------------------------------------------------------------|----|
| 4.1 | Trigger trg_limit_2_riders — Giới hạn 2 tay đua/đội/chặng . . . . .   | 13 |
| 4.2 | Trigger kiểm tra tính hợp lệ của thời gian kết thúc . . . . .         | 14 |
| 4.3 | Stored Procedure sp_calculate_points — Tính điểm F1 tự động . . . . . | 15 |
| 4.4 | View v_race_performance — Nguồn dữ liệu gốc cho BXH . . . . .         | 16 |
| 4.5 | View v_driver_standings — BXH tay đua toàn mùa giải . . . . .         | 16 |
| 4.6 | View v_team_standings — BXH đội đua toàn mùa giải . . . . .           | 16 |
| 6.1 | DDL — Tạo toàn bộ cấu trúc bảng . . . . .                             | 21 |
| 6.2 | INSERT — Dữ liệu mẫu mùa giải F1 2026 . . . . .                       | 22 |
| 6.3 | SELECT với JOIN — Lấy danh sách đăng ký một chặng . . . . .           | 23 |
| 6.4 | SELECT với GROUP BY + HAVING — Đội có điểm trên 50 . . . . .          | 23 |
| 6.5 | INSERT với ON DUPLICATE KEY UPDATE — Upsert kết quả . . . . .         | 23 |
| 6.6 | SELECT với Subquery — BXH đến một chặng cụ thể . . . . .              | 24 |
| 6.7 | Tạo 4 index bổ sung tối ưu hiệu năng . . . . .                        | 24 |

# CHƯƠNG 1: GIỚI THIỆU

## 1.1 Tổng quan về DBMS

Hệ Quản Trị Cơ Sở Dữ Liệu (*Database Management System — DBMS*) là phần mềm trung gian đứng giữa người dùng/ứng dụng và dữ liệu được lưu trữ vật lý, cung cấp các cơ chế để định nghĩa, tạo lập, duy trì và kiểm soát truy cập đến cơ sở dữ liệu. Thay vì mỗi ứng dụng tự quản lý file riêng lẻ — dẫn đến trùng lặp dữ liệu, không nhất quán và khó bảo trì — DBMS cung cấp một kho dữ liệu tập trung, nhất quán và có thể truy cập đồng thời bởi nhiều người dùng.

Các DBMS hiện đại như MySQL, PostgreSQL, Oracle, SQL Server đều xây dựng trên nền tảng mô hình quan hệ (*Relational Model*) được Edgar F. Codd đề xuất năm 1970. Trong mô hình này, dữ liệu được tổ chức thành các bảng (*table*) gồm nhiều hàng (*row*) và cột (*column*), các bảng liên kết với nhau qua khóa ngoại (*foreign key*), và ngôn ngữ SQL (*Structured Query Language*) được dùng để thao tác dữ liệu một cách khai báo (*declarative*).

## 1.2 Vai trò của DBMS trong hệ thống thông tin

Trong kiến trúc hệ thống thông tin hiện đại, DBMS đóng vai trò là lớp nền tảng không thể thiếu:

- **Tính toàn vẹn dữ liệu (*Data Integrity*):** DBMS tự động kiểm tra và đảm bảo dữ liệu luôn hợp lệ thông qua các ràng buộc (*constraints*), trigger và transaction.
- **Tính nhất quán (*Consistency*):** Nhiều người dùng truy cập đồng thời nhưng mỗi người đều nhìn thấy trạng thái nhất quán của dữ liệu, nhờ cơ chế ACID transaction và MVCC (*Multi-Version Concurrency Control*).
- **Tính bảo mật (*Security*):** DBMS cung cấp hệ thống phân quyền chi tiết — mỗi user chỉ truy cập được những gì được cho phép.
- **Tính hiệu năng (*Performance*):** Index, query optimizer và caching giúp truy xuất dữ liệu nhanh ngay cả khi bảng có hàng triệu dòng.
- **Tính bền vững (*Durability*):** Dữ liệu được đảm bảo không bị mất sau khi commit, kể cả khi server gặp sự cố đột ngột.

## 1.3 Giới thiệu bài toán — F1 Championship Management System

Formula 1 (F1) là giải đua xe thể thức một hàng đầu thế giới, với hàng chục đội đua, hơn 20 tay đua chuyên nghiệp và lịch thi đấu kéo dài gần một năm. Mỗi mùa giải gồm nhiều chặng đua (*Grand Prix*) tại các quốc gia khác nhau.

**F1 Championship Management System** là hệ thống phần mềm quản lý toàn bộ vòng đời dữ liệu của một mùa giải F1:

- Quản lý thông tin đội đua, tay đua, hợp đồng ký kết
- Quản lý lịch thi đấu và các chặng đua
- Cho phép đăng ký tay đua vào từng chặng đua
- Ghi nhận kết quả thi đấu sau mỗi chặng
- Tự động tính điểm theo hệ thống tính điểm F1 quốc tế
- Cung cấp bảng xếp hạng (BXH) tay đua và đội đua theo thời gian thực

Hệ thống là ứng dụng web full-stack gồm: Frontend React.js (giao diện người dùng), Backend Node.js/Express (REST API), và MySQL/MariaDB (cơ sở dữ liệu).

## 1.4 Mục tiêu báo cáo

Báo cáo này nhằm:

1. Trình bày quá trình phân tích, thiết kế và triển khai cơ sở dữ liệu cho hệ thống F1 Championship Management.
2. Minh họa cách áp dụng các khái niệm DBMS học thuật (chuẩn hóa, ràng buộc toàn vẹn, transaction ACID, index, view, stored procedure, trigger) vào bài toán thực tế.
3. Phân tích chi tiết nghiệp vụ logic và cách cơ sở dữ liệu hỗ trợ các quy trình nghiệp vụ đó.
4. Đánh giá các quyết định thiết kế và gợi ý hướng cải thiện.



# CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

## 2.1 Khái niệm DBMS

**Định nghĩa:** DBMS là tập hợp các chương trình cho phép người dùng tạo lập, duy trì và truy xuất cơ sở dữ liệu. DBMS cung cấp giao diện giữa người dùng và cơ sở dữ liệu, đồng thời quản lý tất cả truy cập vào dữ liệu.

**Các thành phần ngôn ngữ chính của DBMS:**

- **DDL (Data Definition Language):** Ngôn ngữ định nghĩa cấu trúc — [CREATE TABLE](#), [ALTER TABLE](#), [DROP TABLE](#)
- **DML (Data Manipulation Language):** Ngôn ngữ thao tác dữ liệu — [SELECT](#), [INSERT](#), [UPDATE](#), [DELETE](#).
- **DCL (Data Control Language):** Ngôn ngữ kiểm soát truy cập — [GRANT](#), [REVOKE](#).
- **TCL (Transaction Control Language):** Ngôn ngữ kiểm soát giao dịch — [BEGIN](#), [COMMIT](#), [ROLLBACK](#).

## 2.2 Các mô hình dữ liệu

**Mô hình quan hệ (Relational Model)** — được sử dụng trong dự án này:

Dữ liệu được tổ chức thành các quan hệ (*relation*), biểu diễn dưới dạng bảng 2 chiều. Mỗi hàng (*tuple*) đại diện cho một thực thể, mỗi cột (*attribute*) đại diện cho một thuộc tính. Mỗi quan hệ giữa các bảng được thiết lập qua Khóa ngoại (*Foreign Key*).

**Mô hình phi quan hệ (NoSQL):**

Gồm nhiều dạng: Document Store (MongoDB), Key-Value Store (Redis), Column Family (Cassandra), Graph Database (Neo4j). Phù hợp cho dữ liệu phi cấu trúc hoặc yêu cầu mở rộng ngang cực lớn.

**Lý do chọn mô hình quan hệ cho bài toán F1:**

Dữ liệu F1 có cấu trúc rõ ràng, nhiều mối quan hệ phức tạp giữa các thực thể (tay đua — đội — hợp đồng — chặng — kết quả), và yêu cầu ràng buộc toàn vẹn chặt chẽ. Mô hình quan hệ với MySQL/InnoDB đáp ứng đầy đủ các yêu cầu này.

## 2.3 Kiến trúc 3 mức (Three-Level Architecture)

Kiến trúc ANSI/SPARC định nghĩa 3 mức nhìn vào cơ sở dữ liệu:

- **Mức Ngoài:** Mỗi nhóm người dùng nhìn thấy một “view” khác nhau của dữ liệu. Trong dự án F1, trang BXH chỉ thấy điểm và xếp hạng qua `v_driver_standings`.
- **Mức Khái niệm:** Toàn bộ schema — 7 bảng với các mối quan hệ, ràng buộc, trigger, stored procedure. Đây là tầng DBA làm việc.
- **Mức Vật lý:** InnoDB engine tổ chức dữ liệu thực sự trên đĩa theo B+ Tree, quản lý buffer pool, redo log, undo log.

## 2.4 Các khái niệm cơ bản

**Khóa chính (Primary Key — PK):** Cột hoặc tập cột xác định duy nhất mỗi hàng trong bảng. Ví dụ: `driver_code` trong bảng `DRIVERS`.

**Khóa ngoại (Foreign Key — FK):** Cột tham chiếu đến PK của bảng khác. Ví dụ: `RESULTS.entry_id` tham chiếu đến `RACE_ENTRIES.entry_id`.

**Khóa dự tuyển (Candidate Key):** Bất kỳ tập thuộc tính nào có thể xác định duy nhất một hàng. Trong `RACE_ENTRIES`, cặp (`race_code`, `contract_id`) là candidate key.

**Ràng buộc toàn vẹn (Integrity Constraint):**

- **Domain Constraint:** Giới hạn miền giá trị (NOT NULL, CHECK, ENUM)
- **Entity Integrity:** PK không được NULL
- **Referential Integrity:** FK phải tham chiếu đến giá trị PK tồn tại
- **Semantic Integrity:** Ràng buộc nghiệp vụ tùy chỉnh (Trigger)

# CHƯƠNG 3: MÔ TẢ BÀI TOÀN & PHÂN TÍCH YÊU CẦU

## 3.1 Mô tả hệ thống

Hệ thống gồm 4 module chính:

Bảng 3.1: Các module của hệ thống

| Module   | Tên              | Vai trò                            |
|----------|------------------|------------------------------------|
| Module 1 | Register Racing  | Đăng ký tay đua tham gia chặng đua |
| Module 2 | Update Results   | Nhập kết quả thi đấu sau chặng     |
| Module 3 | Driver Standings | Xem bảng xếp hạng tay đua          |
| Module 4 | Team Standings   | Xem bảng xếp hạng đội đua          |

## 3.2 Người dùng hệ thống

**Ban tổ chức / Staff nhập liệu:**

- Đăng ký tay đua vào từng chặng đua (Module 1)
- Nhập kết quả sau khi chặng kết thúc (Module 2)
- Xem bảng xếp hạng để kiểm tra và công bố kết quả (Module 3, 4)

**Quản trị viên hệ thống (DBA):**

- Khởi tạo và bảo trì cơ sở dữ liệu
- Thêm mùa giải, chặng đua, đội đua, tay đua mới
- Theo dõi hiệu năng và backup định kỳ

**Người xem / Khán giả (Read-only):** Xem bảng xếp hạng, không có quyền ghi dữ liệu.

## 3.3 Yêu cầu chức năng (Functional Requirements)

- FR-01** Hệ thống quản lý thông tin các mùa giải F1 (thêm, sửa, xem).
- FR-02** Hệ thống quản lý danh sách đội đua và tay đua, bao gồm thông tin cá nhân và lịch sử hợp đồng.
- FR-03** Hệ thống cho phép đăng ký tay đua vào từng chặng đua, với ràng buộc mỗi đội tối đa 2 tay đua/chặng.
- FR-04** Hệ thống cho phép nhập kết quả thi đấu (thời gian kết thúc, số vòng hoàn thành, trạng thái) cho từng tay đua.
- FR-05** Hệ thống tự động tính điểm theo hệ thống tính điểm F1 quốc tế (P1=25, P2=18, ..., P10=1) ngay sau khi kết quả được lưu.
- FR-06** Hệ thống cung cấp bảng xếp hạng tay đua và đội đua theo thời gian thực, hỗ trợ lọc theo từng chặng cụ thể.
- FR-07** Hệ thống cho phép xem lịch sử kết quả chi tiết của từng tay đua và từng đội qua các chặng.
- FR-08** Hệ thống hỗ trợ sửa lại kết quả đã nhập (điều chỉnh sau khi xem xét lại).

### 3.4 Yêu cầu phi chức năng (Non-Functional Requirements)

**Hiệu năng**

Bảng xếp hạng tải trong < 2 giây; Module nhập kết quả phản hồi trong < 3 giây kể cả bước tính điểm.

**Toàn vẹn dữ liệu**

Dữ liệu nhất quán tại mọi thời điểm. Không thể nhập kết quả vi phạm nghiệp vụ.

**Bảo mật** Thông tin đăng nhập DB không hardcode. Mọi truy vấn dùng Prepared Statements chống SQL Injection.

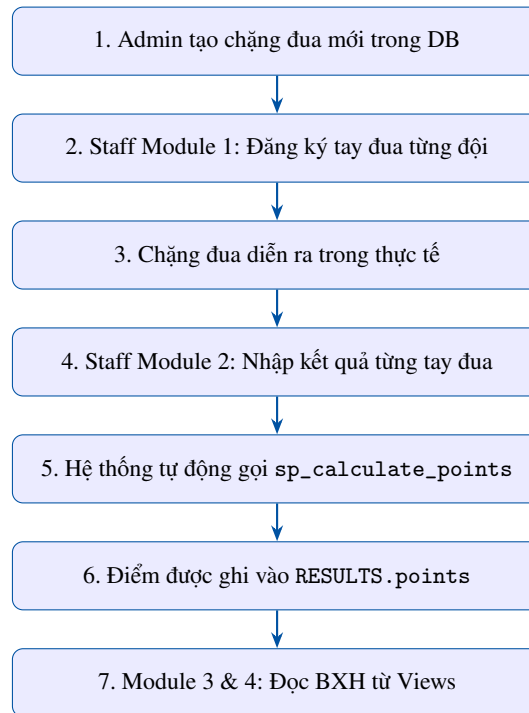
**Khả dụng**

Hỗ trợ nhiều người dùng đồng thời không block nhau (InnoDB MVCC).

**Phục hồi** Chiến lược backup có thể khôi phục dữ liệu đến thời điểm bất kỳ.

# CHƯƠNG 4: LOGIC NGHIỆP VỤ

## 4.1 Quy trình nghiệp vụ tổng thể



Hình 4.1: Vòng đời dữ liệu của một chặng đua trong hệ thống

## 4.2 Business Rules (Quy tắc nghiệp vụ)

### BR-01 — Giới hạn tay đua mỗi đội

Mỗi đội đua chỉ được đăng ký tối đa 2 tay đua cho mỗi chặng đua. Ràng buộc này được thực thi bằng **Trigger** tại tầng Database.

### BR-02 — Hệ thống tính điểm F1

Điểm theo thứ hạng thời gian, chỉ tay đua hoàn thành (`status = 'Finished'`) mới được xếp hạng và tính điểm.

### BR-03 — Tính hợp lệ thời gian

`end_time` phải lớn hơn `start_time` của chặng. Thực thi bằng Trigger bảo vệ cả INSERT và UPDATE.

### BR-04 — Xếp hạng khi bằng điểm

Ưu tiên: (1) Tổng điểm giảm dần; (2) Tổng thời gian tăng dần.

### BR-05 — Điểm đội

Bảng tổng điểm của TẤT CẢ tay đua của đội tại chặng đó (không lấy trung bình).

### BR-06 — Lịch sử hợp đồng

Khi chuyển đội: hợp đồng cũ `is_active = 0`, INSERT hợp đồng mới với `is_active = 1`.

**Bảng điểm F1 chuẩn (BR-02):**

Bảng 4.1: Hệ thống tính điểm F1

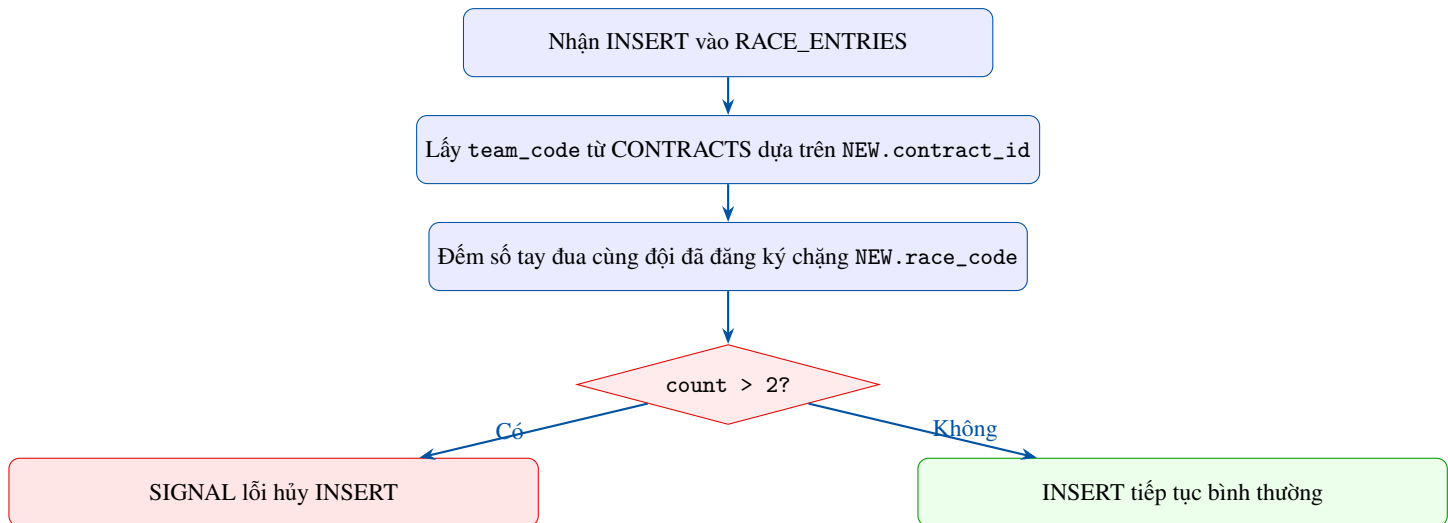
| Hạng          | Điểm | Hạng            | Điểm |
|---------------|------|-----------------|------|
| P1            | 25   | P6              | 8    |
| P2            | 18   | P7              | 6    |
| P3            | 15   | P8              | 4    |
| P4            | 12   | P9              | 2    |
| P5            | 10   | P10             | 1    |
| P11 trở đi: 0 |      | DNF/Accident: 0 |      |

### 4.3 Trigger — Ràng buộc nghiệp vụ tại tầng Database

#### 4.3.1 Trigger 1: trg\_limit\_2\_riders

**Mục đích:** Chặn đăng ký tay đua thứ 3 của cùng một đội vào cùng một chặng.

**Thời điểm kích hoạt:** BEFORE INSERT trên bảng RACE\_ENTRIES.



Hình 4.2: Luồng xử lý của Trigger trg\_limit\_2\_riders

```

1 DELIMITER //
2 CREATE TRIGGER trg_limit_2_riders
3 BEFORE INSERT ON RACE_ENTRIES
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_team_id VARCHAR(10);
7     DECLARE v_count INT;
8     -- Lay team_code của tay đua sắp đăng ký
9     SELECT team_code INTO v_team_id
10    FROM CONTRACTS WHERE contract_id = NEW.contract_id;
11    -- Dem tay đua cùng đội đã đăng ký chặng này
12    SELECT COUNT(*) INTO v_count
13    FROM RACE_ENTRIES re
14    JOIN CONTRACTS c ON re.contract_id = c.contract_id
15    WHERE re.race_code = NEW.race_code
16           AND c.team_code = v_team_id;
17    -- Neu đa dư 2 -> từ chối
  
```

```

18     IF v_count >= 2 THEN
19         SIGNAL SQLSTATE '45000'
20         SET MESSAGE_TEXT = 'Loi: Moi doi chi duoc toi da 2 tay dua!';
21     END IF;
22 END //
23 DELIMITER ;

```

Listing 4.1: Trigger trg\_limit\_2\_riders — Giới hạn 2 tay đua/đội/chặng

#### 4.3.2 Trigger 2 & 3: trg\_check\_time\_insert / trg\_check\_time\_update

**Mục đích:** Đảm bảo end\_time > start\_time của chặng tương ứng.

**Thời điểm kích hoạt:** BEFORE INSERT và BEFORE UPDATE trên bảng RESULTS.

```

1 DELIMITER //
2 CREATE TRIGGER trg_check_time_insert
3 BEFORE INSERT ON RESULTS
4 FOR EACH ROW
5 BEGIN
6     DECLARE v_start_time DATETIME(3);
7     IF NEW.status = 'Finished' AND NEW.end_time IS NOT NULL THEN
8         SELECT r.start_time INTO v_start_time
9         FROM RACE_ENTRIES re
10        JOIN RACES r ON re.race_code = r.race_code
11        WHERE re.entry_id = NEW.entry_id;
12
13        IF NEW.end_time <= v_start_time THEN
14            SIGNAL SQLSTATE '45000'
15            SET MESSAGE_TEXT =
16                'Loi: Thoi gian ket thuc phai lon hon thoi gian bat dau chang!';
17        END IF;
18    END IF;
19 END //
20 DELIMITER ;

```

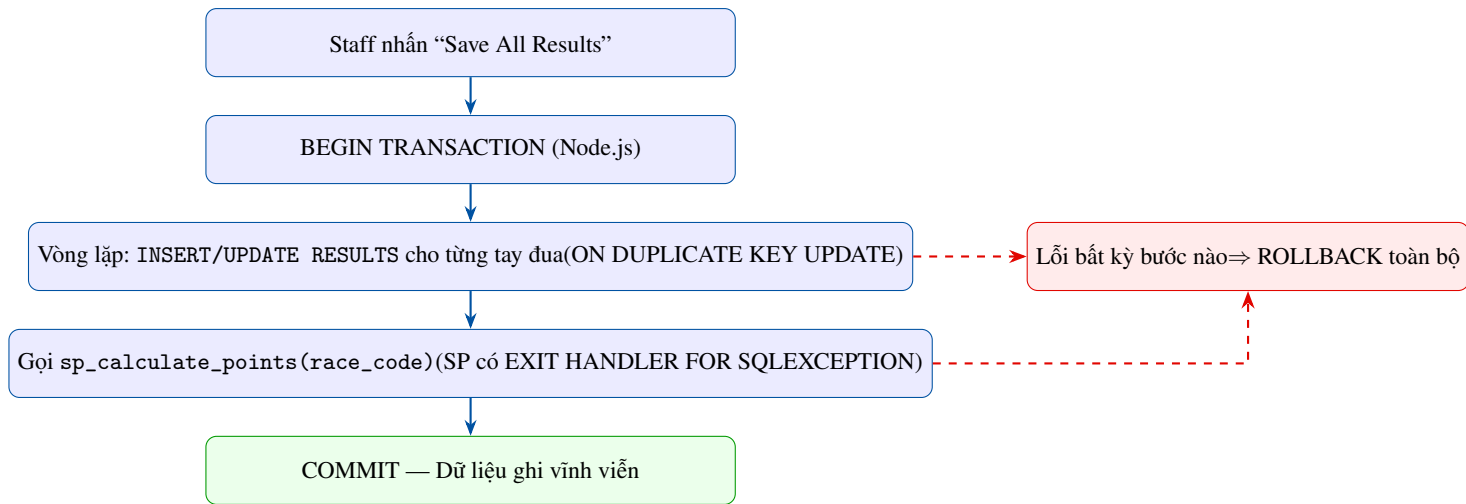
Listing 4.2: Trigger kiểm tra tính hợp lệ của thời gian kết thúc

Trigger trg\_check\_time\_update có cấu trúc hoàn toàn tương tự nhưng áp dụng cho thao tác UPDATE. Cần 2 trigger riêng biệt để bao phủ cả 2 trường hợp nhập mới và chỉnh sửa lại kết quả.

**Lý do đặt ràng buộc ở tầng Database thay vì chỉ ở tầng ứng dụng:** Nếu chỉ kiểm tra ở Frontend hay Backend Node.js, người dùng có thể bypass bằng cách gọi API trực tiếp (Postman, curl) hoặc kết nối DB bằng MySQL Workbench. Trigger đảm bảo ràng buộc được áp dụng tại **MỌI ĐIỂM TRUY CẬP** vào Database.

## 4.4 Transaction — Đảm bảo tính nguyên tử

Đây là nghiệp vụ phức tạp nhất trong hệ thống, cần Transaction để đảm bảo tính ACID:



Hình 4.3: Luồng Transaction của Module 2 — Nhập kết quả

#### 4.5 Stored Procedure — sp\_calculate\_points

**Mục đích:** Tính điểm theo thứ hạng thời gian và lưu vật lý vào RESULTS.points. Được gọi tự động sau mỗi lần lưu kết quả.

```

1  DELIMITER //
2  CREATE PROCEDURE sp_calculate_points(IN p_race_code VARCHAR(10))
3  BEGIN
4      -- Khai báo Handler: Gap loi se tu dong ROLLBACK
5      DECLARE EXIT HANDLER FOR SQLEXCEPTION
6      BEGIN
7          ROLLBACK;
8          RESIGNAL;
9      END;
10
11     START TRANSACTION;
12
13     -- Buoc 1: Reset diem chang ve 0 de tinh lai tu dau
14     UPDATE RESULTS res
15     JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
16     SET res.points = 0
17     WHERE re.race_code = p_race_code;
18
19     -- Buoc 2 & 3: Xep hang thoi gian va cap nhat diem vat ly
20     UPDATE RESULTS target
21     JOIN (
22         SELECT res.entry_id,
23         RANK() OVER (
24             ORDER BY TIMESTAMPDIFF(MICROSECOND, r.start_time, res.end_time) ASC
25         ) AS pos
26         FROM RESULTS res
27         JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
28         JOIN RACES r ON re.race_code = r.race_code
29         WHERE re.race_code = p_race_code AND res.status = 'Finished'
30     ) rnk ON target.entry_id = rnk.entry_id
  
```



```

31     SET target.points = CASE
32         WHEN rnk.pos = 1 THEN 25 WHEN rnk.pos = 2 THEN 18
33         WHEN rnk.pos = 3 THEN 15 WHEN rnk.pos = 4 THEN 12
34         WHEN rnk.pos = 5 THEN 10 WHEN rnk.pos = 6 THEN 8
35         WHEN rnk.pos = 7 THEN 6  WHEN rnk.pos = 8 THEN 4
36         WHEN rnk.pos = 9 THEN 2  WHEN rnk.pos = 10 THEN 1
37         ELSE 0
38     END;
39
40     COMMIT;
41 END //
42 DELIMITER ;

```

Listing 4.3: Stored Procedure sp\_calculate\_points — Tính điểm F1 tự động

**Lưu ý kỹ thuật:** SP dùng **Subquery** (Derived Table trong FROM của UPDATE) thay vì CTE (WITH ... AS (...)) vì MariaDB/MySQL có hạn chế tương thích khi dùng CTE bên trong câu UPDATE. Kết quả hoàn toàn tương đương.

## 4.6 Views — Bảng ảo tổng hợp BXH

```

1 CREATE VIEW v_race_performance AS
2 SELECT re.race_code, c.driver_code, c.team_code,
3        res.status, res.laps_completed, res.points,
4        TIMESTAMPTDIFF(MICROSECOND, r.start_time, res.end_time) / 1000000
5        AS finish_time_seconds,
6        r.start_time AS race_start_time
7 FROM RESULTS res
8 JOIN RACE_ENTRIES re ON re.entry_id = re.entry_id
9 JOIN RACES r ON re.race_code = r.race_code
10 JOIN CONTRACTS c ON re.contract_id = c.contract_id;

```

Listing 4.4: View v\_race\_performance — Nguồn dữ liệu gốc cho BXH

```

1 CREATE VIEW v_driver_standings AS
2 SELECT d.driver_code, d.name, d.nationality, t.name AS team_name,
3        SUM(v.points) AS total_score,
4        SUM(CASE WHEN v.status = 'Finished'
5              THEN v.finish_time_seconds ELSE 0 END) AS total_season_time
6 FROM DRIVERS d
7 JOIN CONTRACTS c ON d.driver_code = c.driver_code AND c.is_active = 1
8 JOIN TEAMS t ON c.team_code = t.team_code
9 JOIN v_race_performance v ON v.driver_code = d.driver_code
10                        AND v.team_code = t.team_code
11 GROUP BY d.driver_code, d.name, d.nationality, t.name
12 ORDER BY total_score DESC, total_season_time ASC;

```

Listing 4.5: View v\_driver\_standings — BXH tay đua toàn mùa giải

```

1 CREATE VIEW v_team_standings AS
2 SELECT t.team_code, t.name AS team_name, t.brand,
3        SUM(v.points) AS team_total_score,

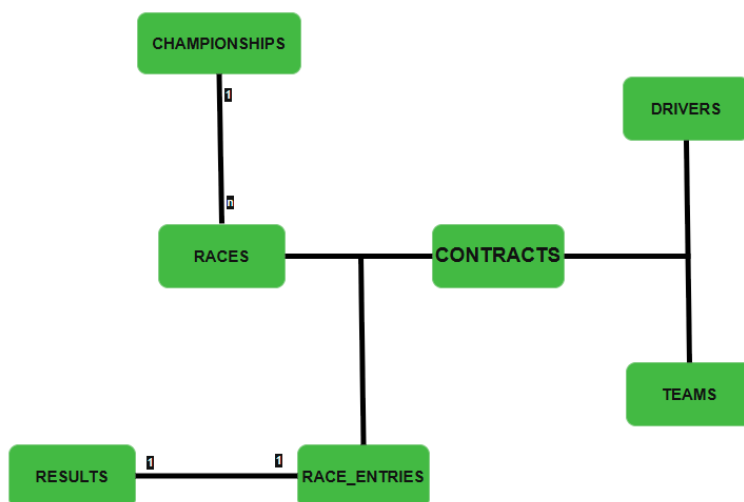
```

```
4         SUM(CASE WHEN v.status = 'Finished'
5              THEN v.finish_time_seconds ELSE 0 END) AS team_total_time
6 FROM TEAMS t
7 JOIN CONTRACTS c ON t.team_code = c.team_code AND c.is_active = 1
8 JOIN v_race_performance v ON v.team_code = t.team_code
9                          AND v.driver_code = c.driver_code
10 GROUP BY t.team_code, t.name, t.brand
11 ORDER BY team_total_score DESC, team_total_time ASC;
```

Listing 4.6: View v\_team\_standings — BXH đội đua toàn mùa giải

# CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU

## 5.1 Mô hình thực thể — liên kết (ERD)



Hình 5.1: Sơ đồ ERD của hệ thống F1 Championship Management

## 5.2 Mô tả chi tiết các thực thể

Bảng 5.1: Bảng CHAMPIONSHIPS — Mùa giải

| Cột         | Kiểu         | Ràng buộc | Mô tả                    |
|-------------|--------------|-----------|--------------------------|
| champ_code  | VARCHAR(10)  | PK        | Mã mùa giải. VD: F1-2026 |
| name        | VARCHAR(255) | NOT NULL  | Tên đầy đủ mùa giải      |
| description | TEXT         | NULL      | Mô tả ngắn               |

Bảng 5.2: Bảng CONTRACTS — Hợp đồng tay đua — đội

| Cột         | Kiểu        | Ràng buộc         | Mô tả                 |
|-------------|-------------|-------------------|-----------------------|
| contract_id | INT         | PK AUTO_INCREMENT | ID hợp đồng           |
| driver_code | VARCHAR(10) | FK → DRIVERS      | Tay đua               |
| team_code   | VARCHAR(10) | FK → TEAMS        | Đội đua               |
| is_active   | TINYINT     | DEFAULT 1         | 1=hiệu lực, 0=hết hạn |

Bảng 5.3: Bảng RESULTS — Kết quả thi đấu

| Cột            | Kiểu        | Ràng buộc             | Mô tả                    |
|----------------|-------------|-----------------------|--------------------------|
| entry_id       | INT         | PK, FK → RACE_ENTRIES | Tham chiếu 1-1           |
| end_time       | DATETIME(3) | NULL                  | Thời điểm về đích        |
| laps_completed | INT         | CHECK ≥ 0             | Số vòng hoàn thành       |
| status         | ENUM        | DEFAULT 'Finished'    | Trạng thái kết thúc      |
| points         | INT         | DEFAULT 0, CHECK ≥ 0  | Điểm F1 (do SP cập nhật) |

### 5.3 Mô hình quan hệ

```

CHAMPIONSHIPS(champ_code, name, description)
TEAMS(team_code, name, brand, owner, description)
DRIVERS(driver_code, name, date_of_birth, nationality)
CONTRACTS(contract_id, driver_code*, team_code*, is_active)
RACES(race_code, name, num_laps, start_time, champ_code*)
RACE_ENTRIES(entry_id, race_code*, contract_id*)
RESULTS(entry_id*, end_time, laps_completed, status, points)

* Dấu * biểu thị khóa ngoại

```

### 5.4 Chuẩn hóa (Normalization)

#### 5.4.1 1NF — First Normal Form

**Yêu cầu:** Mỗi cột chứa giá trị nguyên tử, không có nhóm lặp, có PK.

**Đánh giá:** Tất cả 7 bảng đạt 1NF. Ví dụ: danh sách tay đua của một đội không được lưu trong một cột text dạng "VER,NOR,PIA" mà được lưu thành nhiều dòng riêng trong bảng CONTRACTS.

#### 5.4.2 2NF — Second Normal Form

**Yêu cầu:** Đạt 1NF + không có phụ thuộc hàm bộ phận.

**Đánh giá:** Tất cả bảng đạt 2NF. RACE\_ENTRIES dùng surrogate key entry\_id thay vì khóa ghép (race\_code, contract\_id), loại bỏ khả năng vi phạm 2NF nếu thêm thuộc tính mới.

#### 5.4.3 3NF — Third Normal Form

**Yêu cầu:** Đạt 2NF + không có phụ thuộc hàm bắc cầu.

**Ví dụ vi phạm đã được xử lý:** Nếu để cột team\_name trong bảng RACE\_ENTRIES:

$$\text{entry\_id} \rightarrow \text{contract\_id} \rightarrow \text{team\_code} \rightarrow \text{team\_name}$$

team\_name phụ thuộc bắc cầu qua contract\_id → team\_code. Giải pháp: loại bỏ, JOIN qua CONTRACTS → TEAMS khi cần.

#### 5.4.4 BCNF — Boyce-Codd Normal Form

**Yêu cầu:** Mọi phụ thuộc hàm  $X \rightarrow Y$  đều có X là superkey.

**Đánh giá:** Không có vi phạm. Trong CONTRACTS: contract\_id → (driver\_code, team\_code, is\_active) — contract\_id là PK (superkey). Trong RESULTS: entry\_id → (end\_time, status, points) — entry\_id là PK.

**Kết luận:** Schema đạt chuẩn BCNF, cân bằng tốt giữa tính chuẩn hóa và hiệu năng thực tế.

# CHƯƠNG 6: TRIỂN KHAI

## 6.1 Môi trường và công nghệ

Bảng 6.1: Bảng công nghệ sử dụng

| Thành phần  | Công nghệ         | Phiên bản   |
|-------------|-------------------|-------------|
| DBMS        | MySQL / MariaDB   | 8.0 / 10.6+ |
| Backend     | Node.js + Express | 18+         |
| DB Driver   | mysql2/promise    | 3.x         |
| Frontend    | React.js          | 18+         |
| Quản trị DB | MySQL Workbench   | 8.x         |

## 6.2 Schema — Tạo cấu trúc bảng (DDL)

```

1 DROP DATABASE IF EXISTS F1_Championship_Management;
2 CREATE DATABASE F1_Championship_Management;
3 USE F1_Championship_Management;
4
5 CREATE TABLE CHAMPIONSHIPS (
6     champ_code VARCHAR(10) PRIMARY KEY,
7     name        VARCHAR(255) NOT NULL,
8     description TEXT
9 );
10
11 CREATE TABLE TEAMS (
12     team_code   VARCHAR(10) PRIMARY KEY,
13     name        VARCHAR(100) NOT NULL,
14     brand       VARCHAR(100),
15     owner       VARCHAR(100),
16     description TEXT
17 );
18
19 CREATE TABLE DRIVERS (
20     driver_code VARCHAR(10) PRIMARY KEY,
21     name        VARCHAR(100) NOT NULL,
22     date_of_birth DATE,
23     nationality  VARCHAR(50),
24     biography    TEXT
25 );
26
27 CREATE TABLE CONTRACTS (
28     contract_id INT AUTO_INCREMENT PRIMARY KEY,
29     driver_code VARCHAR(10),
30     team_code   VARCHAR(10),
31     is_active   TINYINT DEFAULT 1,
32     FOREIGN KEY (driver_code) REFERENCES DRIVERS(driver_code),

```

```

33     FOREIGN KEY (team_code) REFERENCES TEAMS(team_code)
34 );
35
36 CREATE TABLE RACES (
37     race_code    VARCHAR(10) PRIMARY KEY,
38     name         VARCHAR(100) NOT NULL,
39     num_laps     INT          NOT NULL,
40     location     VARCHAR(255),
41     start_time   DATETIME(3),
42     champ_code   VARCHAR(10),
43     description  TEXT,
44     FOREIGN KEY (champ_code) REFERENCES CHAMPIONSHIPS(champ_code)
45 );
46
47 CREATE TABLE RACE_ENTRIES (
48     entry_id     INT AUTO_INCREMENT PRIMARY KEY,
49     race_code    VARCHAR(10),
50     contract_id  INT,
51     FOREIGN KEY (race_code) REFERENCES RACES(race_code),
52     FOREIGN KEY (contract_id) REFERENCES CONTRACTS(contract_id),
53     UNIQUE(race_code, contract_id)
54 );
55
56 CREATE TABLE RESULTS (
57     entry_id     INT PRIMARY KEY,
58     end_time     DATETIME(3),
59     laps_completed INT,
60     status       ENUM('Finished', 'DNF', 'Accident') DEFAULT 'Finished',
61     points       INT DEFAULT 0,
62     FOREIGN KEY (entry_id) REFERENCES RACE_ENTRIES(entry_id),
63     CONSTRAINT chk_laps_completed CHECK (laps_completed >= 0),
64     CONSTRAINT chk_points          CHECK (points >= 0)
65 );

```

Listing 6.1: DDL — Tạo toàn bộ cấu trúc bảng

### 6.3 Nạp dữ liệu mẫu (DML — INSERT)

```

1  INSERT INTO CHAMPIONSHIPS VALUES ('F1-2026', 'Formula 1 2026', 'Season');
2
3  INSERT INTO TEAMS VALUES
4      ('RBR', 'Red Bull Racing', 'Red Bull', 'Christian Horner', ''),
5      ('FER', 'Scuderia Ferrari', 'Ferrari', 'Fred Vasseur', ''),
6      ('MER', 'Mercedes-AMG', 'Mercedes', 'Toto Wolff', ''),
7      ('MCL', 'McLaren', 'McLaren', 'Zak Brown', '');
8
9  INSERT INTO DRIVERS VALUES
10     ('VER', 'Max Verstappen', '1997-09-30', 'Dutch', ''),
11     ('NOR', 'Lando Norris', '1999-11-13', 'British', ''),
12     ('PIA', 'Oscar Piastri', '2001-04-06', 'Australian', '');

```

```

13      ('LEC', 'Charles Leclerc', '1997-10-16', 'Monegasque', ''),
14      ('HAM', 'Lewis Hamilton', '1985-01-07', 'British', '');
15
16 INSERT INTO CONTRACTS (driver_code, team_code) VALUES
17      ('VER', 'RBR'), ('NOR', 'MCL'), ('PIA', 'MCL'),
18      ('LEC', 'FER'), ('HAM', 'MER');
19
20 INSERT INTO RACES VALUES
21      ('BHR26', 'Bahrain GP', 57, 'Sakhir',
22          '2026-03-05 15:00:00.000', 'F1-2026', ''),
23      ('JED26', 'Saudi Arabian GP', 50, 'Jeddah',
24          '2026-03-19 17:00:00.000', 'F1-2026', ''),
25      ('MONACO26', 'Monaco GP', 78, 'Monte Carlo',
26          '2026-05-24 15:00:00.000', 'F1-2026', '');

```

Listing 6.2: INSERT — Dữ liệu mẫu mùa giải F1 2026

## 6.4 Các câu truy vấn SQL quan trọng

```

1 SELECT re.entry_id, d.name AS driver_name,
2        t.name AS team_name, d.nationality
3 FROM RACE_ENTRIES re
4 JOIN CONTRACTS c ON re.contract_id = c.contract_id
5 JOIN DRIVERS d ON c.driver_code = d.driver_code
6 JOIN TEAMS t ON c.team_code = t.team_code
7 WHERE re.race_code = 'MONACO26'
8 ORDER BY t.name ASC, d.name ASC;

```

Listing 6.3: SELECT với JOIN — Lấy danh sách đăng ký một chặng

```

1 SELECT t.team_code, t.name AS team_name,
2        SUM(vp.points) AS team_score
3 FROM TEAMS t
4 JOIN CONTRACTS c ON t.team_code = c.team_code AND c.is_active = 1
5 JOIN v_race_performance vp ON vp.team_code = t.team_code
6 GROUP BY t.team_code, t.name
7 HAVING SUM(vp.points) > 50
8 ORDER BY team_score DESC;

```

Listing 6.4: SELECT với GROUP BY + HAVING — Đội có điểm trên 50

```

1 INSERT INTO RESULTS (entry_id, end_time, laps_completed, status)
2 VALUES (5, '2026-05-24 17:26:15.483', 78, 'Finished')
3 ON DUPLICATE KEY UPDATE
4     end_time = VALUES(end_time),
5     laps_completed = VALUES(laps_completed),
6     status = VALUES(status);
7 -- points duoc cap nhap tu dong boi sp_calculate_points

```

Listing 6.5: INSERT với ON DUPLICATE KEY UPDATE — Upsert kết quả



```

1 SELECT d.driver_code, d.name, t.name AS team_name,
2        SUM(vp.points) AS total_score,
3        SUM(CASE WHEN vp.status = 'Finished'
4              THEN vp.finish_time_seconds ELSE 0 END) AS total_time
5 FROM DRIVERS d
6 JOIN CONTRACTS c ON d.driver_code = c.driver_code AND c.is_active = 1
7 JOIN TEAMS t ON c.team_code = t.team_code
8 JOIN v_race_performance vp ON vp.driver_code = d.driver_code
9 JOIN RACES r ON vp.race_code = r.race_code
10 WHERE r.start_time <= (
11     SELECT start_time FROM RACES WHERE race_code = 'JED26'
12 )
13 GROUP BY d.driver_code, d.name, t.name
14 ORDER BY total_score DESC, total_time ASC;

```

Listing 6.6: SELECT với Subquery — BXH đến một chặng cụ thể

## 6.5 Index — Tối ưu hóa hiệu năng

```

1 -- Tang toc truy van Module 2 va Views BXH
2 CREATE INDEX idx_race_code ON RACE_ENTRIES(race_code);
3
4 -- Covering Index cho Join theo doi
5 CREATE INDEX idx_team_code_contracts ON CONTRACTS(team_code, driver_code);
6
7 -- Tang toc WHERE status='Finished' + ORDER BY trong Window Function
8 CREATE INDEX idx_results_status_time ON RESULTS(status, end_time);
9
10 -- Tang toc loc BXH theo moc thoi gian chang
11 CREATE INDEX idx_races_start_time ON RACES(start_time);

```

Listing 6.7: Tạo 4 index bổ sung tối ưu hiệu năng

Bảng 6.2: Chiến lược Index của hệ thống

| Index                   | Bảng.Cột                          | Phục vụ truy vấn                      |
|-------------------------|-----------------------------------|---------------------------------------|
| idx_race_code           | RACE_ENTRIES(race_code)           | WHERE race_code = ? — dùng nhiều nhất |
| idx_team_code_contracts | CONTRACTS(team_code, driver_code) | JOIN + Trigger đếm tay đua            |
| idx_results_status_time | RESULTS(status, end_time)         | RANK() ORDER BY trong SP              |
| idx_races_start_time    | RACES(start_time)                 | BXH lọc theo mốc thời gian            |

# CHƯƠNG 7: ĐÁNH GIÁ

## 7.1 Đánh giá thiết kế cơ sở dữ liệu

### 7.1.1 Điểm mạnh

**Chuẩn hóa đạt BCNF:** Schema đạt chuẩn BCNF với thiết kế tách bạch rõ ràng. Không có dữ liệu dư thừa — thông tin đội chỉ trong TEAMS, thông tin tay đua chỉ trong DRIVERS.

**Mô hình hóa đúng thực tế nghiệp vụ:** CONTRACTS phản ánh đúng bản chất hợp đồng trong F1 (lịch sử chuyển đội, is\_active). RACE\_ENTRIES tách biệt với RESULTS phản ánh đúng hai sự kiện có thời điểm khác nhau.

**Phân tầng ràng buộc rõ ràng:**

- Tầng DB: NOT NULL, UNIQUE, FK, CHECK, TRIGGER — không bỏ qua được
- Tầng Backend: Validation server-side, Prepared Statements
- Tầng Frontend: UX gợi ý và validation trước khi submit

**Sử dụng đầy đủ các đối tượng DBMS:**

Bảng 7.1: Tổng hợp các đối tượng DBMS trong hệ thống

| Đối tượng        | Số lượng   | Mục đích                |
|------------------|------------|-------------------------|
| Table            | 7          | Lưu trữ dữ liệu căn bản |
| View             | 3          | Đóng gói truy vấn BXH   |
| Stored Procedure | 1          | Logic tính điểm F1      |
| Trigger          | 3          | Ràng buộc nghiệp vụ     |
| Index            | 4          | Tối ưu hiệu năng        |
| Transaction      | Nhiều điểm | Đảm bảo tính ACID       |

## 7.2 Hạn chế hiện tại

### H1 — Không lưu ngày bắt đầu/kết thúc hợp đồng

CONTRACTS chỉ có is\_active (0/1), không có start\_date và end\_date. Không thể biết tay đua chuyển đội vào ngày cụ thể nào. *Cải thiện:* ALTER TABLE CONTRACTS ADD COLUMN start\_date DATE, ADD COLUMN end\_date DATE;

### H2 — Race Condition tại Trigger

Nếu 2 nhân viên đồng thời đăng ký tay đua thứ 2 của cùng đội, cả 2 transaction đọc count = 1 và đều pass trigger. Kết quả: đội có 3 tay đua. *Giải pháp:* Dùng SELECT ... FOR UPDATE để lock dòng trước khi kiểm tra.

### H3 — Chưa có Authentication/Authorization

Hệ thống hiện không có đăng nhập và phân quyền. Mọi người đều có thể nhập kết quả.

### H4 — Chưa có Audit Log

Không ghi lại lịch sử ai đã thay đổi dữ liệu. Cần thêm bảng AUDIT\_LOG + Trigger ghi lại thay đổi.

### 7.3 Khả năng mở rộng

- **Ngắn hạn:** Thêm Authentication (JWT), giải quyết Race Condition, thêm Audit Log, thêm ngày bắt đầu/kết thúc hợp đồng.
- **Dài hạn:** Hỗ trợ nhiều mùa giải với bảng SCORING\_SYSTEMS theo từng era F1. Export BXH PDF/Excel. Tích hợp MySQL Replication (Read Replica) để tách tải đọc và ghi.

# CHƯƠNG 8: KẾT LUẬN

## 8.1 Tóm tắt kết quả

Báo cáo đã trình bày toàn diện quá trình thiết kế và triển khai hệ thống F1 Championship Management với MySQL/MariaDB. Kết quả đạt được:

- Schema 7 bảng đạt chuẩn BCNF, phản ánh đúng thực tế nghiệp vụ F1.
- 6 Foreign Key đảm bảo toàn vẹn tham chiếu hoàn chỉnh trong chuỗi dữ liệu.
- 3 Trigger thực thi ràng buộc nghiệp vụ tại tầng DB (giới hạn 2 tay đua/đội, thời gian hợp lệ).
- 1 Stored Procedure với Transaction ACID để tính điểm F1 tự động và nhất quán.
- 3 View đóng gói logic truy vấn phức tạp, API chỉ cần 1 câu SELECT.
- 4 Index bổ sung tối ưu hiệu năng cho các truy vấn quan trọng nhất.

## 8.2 Bài học rút ra

1. **Database là nguồn sự thật cuối cùng (Single Source of Truth):** Constraints và Trigger ở DB không thể bị bypass từ bất kỳ hướng nào — đây là lớp bảo vệ quan trọng nhất.
2. **Thiết kế schema chuẩn hóa tốt ngay từ đầu:** Tách RESULTS khỏi RACE\_ENTRIES, dùng CONTRACTS thay vì lưu team\_code trong DRIVERS giúp tránh hàng loạt vấn đề nhất quán dữ liệu về sau.
3. **Transaction cho mọi nghiệp vụ đa bước:** Bất kỳ nghiệp vụ nào liên quan đến nhiều bảng đều cần Transaction.
4. **Window Function RANK() là công cụ mạnh:** Giải quyết bài toán xếp hạng thanh lịch và hiệu quả  $O(n \log n)$  so với subquery đếm  $O(n^2)$ .
5. **Index cần chọn lọc, không thừa:** Chỉ tạo index trên các cột xuất hiện trong WHERE, JOIN, ORDER BY của truy vấn quan trọng.
6. **Tính toán đúng tầng:** Đặt logic tính điểm vào Stored Procedure (tầng DB) thay vì code Node.js giúp DB tự bảo vệ tính nhất quán.

## 8.3 Hướng phát triển

- Thêm JWT Authentication và role-based access control
- Giải quyết Race Condition bằng SELECT ... FOR UPDATE
- Thêm bảng lưu lịch sử thay đổi dữ liệu (Audit Log)
- Hỗ trợ nhiều hệ thống tính điểm theo từng era lịch sử F1
- Tích hợp Read Replica để scale khi traffic tăng cao

# TÀI LIỆU THAM KHẢO

1. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
2. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
4. MariaDB Foundation. (2024). *MariaDB Server Documentation*. <https://mariadb.com/kb/en/documentation/>
5. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
6. Fédération Internationale de l'Automobile (FIA). (2024). *Formula 1 Sporting Regulations*. <https://www.formula1.com/>